
System Design Mastery Notes

From basics to senior-level depth

Chapter 1 — Computing Foundations, Concurrency & the Node.js Runtime

Notes by Amjesh Kumar Singh • Restructured & deepened edition (draft for review)

★ **MUST REMEMBER** — How to read this book

- **Yellow highlight** = a concept you must be able to explain in an interview.
- ★ **Must Remember** boxes = core rules. ♦ **Senior Insight** = depth expected at senior/staff level.
- ✖ **Common Mistake** = traps people fall into. ⚡ **Quick Recall** = last-minute revision lines.
- 📝 **Correction** = places where I fixed or refined something from your original notes.

Proposed book roadmap (your notes, regrouped in the best learning order)

Part	Ch	Chapter	Covers (from your notes + added depth)
I. Foundations	1	Computing, Concurrency & Node.js runtime (this PDF)	CPU, process, thread, context switch, scheduling, concurrency vs parallelism, multithreading vs async, V8, libuv, event loop, cluster
	2	Memory, Storage & Numbers	RAID, latency numbers, powers of two, back-of-envelope estimation, SSD vs HDD
	3	Networking Fundamentals	OSI, MAC/IP/NAT/Port/Socket, 5-tuple, TCP vs UDP, DNS, SSH, VPN, latency/bandwidth/throughput
	4	Communication & APIs	HTTP verbs, REST, RPC/gRPC, GraphQL, WebSocket, idempotency, API versioning
II. Scalability & Reliability	5	Scalability, Availability & Consistency	Vertical/horizontal, CAP, PACELC, consistency patterns, failover, nines of availability
	6	DNS & CDN at scale	DNS records/routing policies, push vs pull CDN, cache invalidation
	7	Load Balancer & Reverse Proxy	L4 vs L7, algorithms, health checks, sticky sessions, SSL termination, Nginx/HAProxy
	8	Application Layer & Microservices	Service discovery, API gateway, service mesh, monolith vs microservices
III. Data	9	Relational Databases	ACID, indexes, replication, federation, sharding, denormalization, SQL tuning
	10	NoSQL & Choosing a Database	KV, document, wide-column, graph, BASE, SQL vs NoSQL decision guide
IV. Building Blocks	11	Caching	Cache-aside, write-through/behind, refresh-ahead, eviction, stampede, Redis vs Memcached
	12	Asynchronism & Messaging	Queues, task queues, back pressure, Kafka vs RabbitMQ vs SQS, delivery guarantees
	13	Security Essentials	Encryption, XSS, SQLi, AuthN/AuthZ, OAuth/JWT, rate limiting, OWASP
V. Mastery	14	Advanced Distributed Systems <i>(new)</i>	Consistent hashing, consensus (Raft), distributed transactions, saga, observability
	15	Interview Framework & Case Studies	Design process, 20+ classic designs, real-world architectures (Twitter etc.)

Chapter 1

Computing Foundations, Concurrency & the Node.js Runtime

Why this chapter comes first: every system design decision — threads vs async, how many servers, why a cache helps — comes back to one problem: **a few scarce resources (CPU cores, memory, disk, network) must serve a huge number of tasks**. If you understand how one machine runs many tasks, you understand why distributed systems look the way they do.

🔗 QUICK RECALL — Chapter map

- 1.1 CPU core → 1.2 Process → 1.3 Thread → 1.4 Context switching → 1.5 Scheduling
- 1.6 Concurrency vs Parallelism → 1.7 Workload types & I/O models → 1.8 Multithreading vs Async
- 1.9 Node.js: V8 + libuv → 1.10 Thread pool → 1.11 Event loop phases → 1.12 Blocking the loop → 1.13 Scaling Node
- 1.14 Real-world: 1000 clients → 1.15 Decision guide → 1.16 Interview Q&A + Cheat sheet

1.1 CPU Core — the physical worker

A **CPU core** is a physical execution unit. At any single instant, **one core runs exactly one thread**. Everything that looks like “many things at once” on a 1-core machine is just fast switching.

Term	Meaning	Example
Physical core	Real execution hardware	8-core CPU = 8 threads truly running at the same moment
Logical core (hyper-threading / SMT)	Core presents 2 hardware threads that share its execution units; gives ~20-30% extra throughput, not 2x	8 cores / 16 logical CPUs
Clock speed	Cycles per second (GHz). Today's gains come from more cores, not higher GHz	3.5 GHz

★ MUST REMEMBER

- 1 core = 1 worker = **1 thread at a time**. N cores = at most N threads in *true* parallel.
- Hyper-threading helps when a thread stalls (e.g., waiting for memory) — it is not double the cores.

1.2 Process — the container

A **process** is a running program. The OS gives it its own **isolated virtual address space** (code, heap, stack, open files) and at least one thread. Process A cannot read process B's memory — the OS and CPU (MMU) enforce this.

Inside a process:

```
+----- PROCESS -----+
| Program code   | Heap (shared by all threads) |
| Globals / data | Open files, sockets (file descriptors) |
+-----+-----+
| Thread 1      | Thread 2      | Thread 3      |
| own stack     | own stack     | own stack     |
| own registers | own registers | own registers |
| own program ctr | own program ctr | own program ctr |
+-----+-----+-----+
```

Process vs Thread

Aspect	Process	Thread
Memory	Own isolated address space	Shares heap, globals, files, sockets with sibling threads
Creation cost	Heavy (new address space, page tables)	Light (only new stack + registers)
Communication	IPC: pipes, sockets, shared memory, message queues	Direct shared memory (needs synchronization)
Failure impact	Isolated — one crash does not kill others	A fatal fault in one thread can kill the whole process
Context switch	More expensive (address space change, TLB flush)	Cheaper (same address space)
Typical use	Isolation & security (Chrome tabs, Nginx workers, Node cluster)	Parallelism inside one app (DB engines, JVM servers)

★ MUST REMEMBER

- Process = Isolated, Thread = Shared. This single sentence explains almost every trade-off: isolation gives safety but costs communication; sharing gives speed but costs correctness.

1.3 Thread — the smallest unit of execution

A **thread** is the smallest unit the OS schedules onto a core.

Each thread has its OWN	Threads SHARE (danger zone)
Call stack CPU registers Instruction pointer (current line) Thread-local storage	Heap memory Global / static variables Open files Sockets (file descriptors)

1.3.1 Race condition — the core bug of shared memory

A race condition happens when the result depends on the **timing** of threads accessing shared data. Even `counter++` is not one step — it is **read → add → write**.

```
Account balance = 100. Two threads each withdraw 80:
```

```
Thread A: if (balance >= 80)  -> true
Thread B: if (balance >= 80)  -> true    (A has not subtracted yet!)
Thread A: balance = balance-80 -> 20
Thread B: balance = balance-80 -> -60    <-- overdraft, bug
```

```
Fix: make "check + subtract" ONE atomic critical section (lock / atomic / DB row lock).
```

1.3.2 Tools & their cost

Tool	What it does	Risk
Mutex / lock	Only one thread inside the critical section	Contention, deadlock
Semaphore	Allows up to N threads at once (e.g., connection pool)	Mis-counting → leaks
Atomic ops / CAS	Lock-free single-variable updates	Hard to reason about for complex data
Immutable data / message passing	Avoid sharing at all (Go channels, actors, Node workers)	Copy overhead

◆ **SENIOR INSIGHT** — **Deadlock = 4 conditions (Coffman). Break any one and it cannot happen**

- **Mutual exclusion** (resource held exclusively) + **Hold and wait** (hold one, wait for another) + **No preemption** (cannot be forcibly taken) + **Circular wait** (A waits B, B waits A).
- Practical prevention: **always acquire locks in a global order**, use `tryLock` with timeout, keep critical sections tiny.
- Related failures: **starvation** (a thread never gets the lock) and **livelock** (threads keep reacting to each other but make no progress).

1.4 Context switching — the hidden tax

A **context switch** happens when the CPU stops running one thread and starts another. It is required whenever **runnable threads > cores**, or a thread blocks (waiting for I/O, a lock, sleep).

- 1 **Save** current thread state (registers, stack pointer, program counter) into its control block.
- 2 **Pick** next thread (scheduler decision).
- 3 **Load** the next thread's state, and if it belongs to another process, switch address space (TLB flush).
- 4 **Resume** execution.

Cost type	Explanation
Direct cost	Saving/restoring state + kernel work: roughly 1-10 µs per switch (order of magnitude).
Indirect cost (bigger!)	CPU caches (L1/L2) and TLB now hold the <i>previous</i> thread's data → cache misses after switching. Main-memory access is ~100 ns vs ~0.5 ns for L1.
Voluntary vs involuntary	Voluntary: thread blocked on I/O/lock. Involuntary: OS preempted it (time slice ended). Many involuntary switches = CPU oversubscribed.

★ **MUST REMEMBER**

- Context switching is **pure overhead** — no useful work is done during it.
- More threads than cores ⇒ more switching ⇒ lower throughput. Async/event-loop switching is a plain function call (tens of ns), not a kernel switch.

1.5 Scheduling — who runs, who waits, how long

The **scheduler** decides which thread runs next on limited cores. With 4-64 cores but thousands of threads, this decision happens thousands of times per second.

Model	How it works	Pros / Cons	Where
Preemptive	OS can <i>forcibly</i> pause a running thread when its time slice ends or a higher-priority thread arrives. The program does not choose when to stop.	Fair, one bad thread cannot hog CPU / but needs locks, and switches are costly	Linux, Windows, macOS threads
Cooperative	A task runs until it <i>voluntarily</i> yields (await, I/O wait).	Cheap switching, no data races / but one task that never yields freezes everyone	JS event loop, Python asyncio, Go (mostly), coroutines

Common algorithms (know the names): **FCFS** (first come first served), **Round Robin** (equal time slices), **Priority** scheduling, **Multilevel feedback queue**, and Linux's **CFS** (Completely Fair Scheduler: gives each runnable thread a fair share of CPU time). Time-slice trade-off: **short slice** = responsive but more switching; **long slice** = efficient but laggy.

◆ SENIOR INSIGHT — Why this matters in system design

- A server with 2000 busy threads on 8 cores spends a large part of its time scheduling instead of working — this is why thread-per-request does not scale to 10k+ connections.
- Same idea reappears at cluster level: Kubernetes, load balancers and job queues are all “schedulers” deciding who runs where.

1.6 Concurrency vs Parallelism

	Concurrency	Parallelism
Definition	Making progress on multiple tasks during the same time period (they overlap in time)	Executing multiple tasks at the exact same instant
Needs	Good task structure; can work on one core	Multiple cores/workers
About	Dealing with many things (structure / composition)	Doing many things (execution)
Kitchen analogy	One cook: starts boiling water, chops vegetables while waiting, checks the pot again...	Two cooks: one boils water, the other chops — at the same moment
Example	Node.js event loop serving 10k sockets on one thread	Video encoding split across 8 cores

The four combinations

Cores	Tasks	Result
1	1 task	Plain sequential execution
1	Many tasks interleaved	Concurrent, not parallel (Node JS thread)
Many	Many independent tasks	Concurrent AND parallel (thread pool, multi-process server)
Many	One big task split into parts	Parallel (data parallelism, MapReduce style)

◆ SENIOR INSIGHT — Amdahl's Law — why adding cores has a ceiling

- $\text{Speedup} = 1 / ((1 - p) + p / n)$, where p = parallelizable fraction, n = number of cores.
- Example: $p = 90\%$, $n = 8 \rightarrow 1 / (0.10 + 0.1125) \approx 4.7\times$ (not $8\times$). Even with infinite cores the max is $1/(1-p) = 10\times$.
- Lesson: the serial part (locks, a single DB, one master node) is the real bottleneck. This is exactly why we shard, partition and avoid single points of contention in later chapters.

◀ QUICK RECALL

- Concurrency = **structure**. Parallelism = **simultaneous execution**. You can have either without the other.

1.7 Workload types & I/O models

Choosing between threads and async starts with one question: is the task **CPU-bound** or **I/O-bound**?

	CPU-bound	I/O-bound
Time spent	Computing (encoding, hashing, ML, parsing big JSON)	Waiting (DB, network, disk, other APIs)
Bottleneck	CPU cores	Latency of the thing you wait for
Best model	Threads / processes, one per core	Async, non-blocking I/O (event loop)
Thread count rule	$\approx$ number of cores	$\approx \text{cores} \times (1 + \text{wait time} / \text{compute time})$

Blocking vs non-blocking, sync vs async

Blocking: the calling thread sleeps until the operation finishes. **Non-blocking:** the call returns immediately (“not ready yet”) and you check back. **Sync/async** is about *how you get the result* (return value vs callback/promise/notification).

I/O model	How it works	Used by
Blocking I/O	Thread calls <code>read()</code> and sleeps. 1 connection $\approx$ 1 thread.	Classic thread-per-request servers
I/O multiplexing	One thread asks the kernel “tell me which of these 10,000 sockets are ready” via epoll (Linux), kqueue (BSD/macOS), select/poll .	Node.js (libuv), Nginx, Redis
True async I/O	Kernel performs the I/O and notifies on completion: IOCP (Windows), io_uring (modern Linux).	Windows services, newer high-performance servers

◆ SENIOR INSIGHT — The C10K problem

- Early-2000s question: how to serve 10,000 concurrent connections on one machine? Answer: stop using one thread per connection; use readiness notification (epoll/kqueue) + event loop.
- Today the question is C10M; the answer is the same idea plus kernel-bypass and careful tuning. Limits you will hit first: **file descriptors (ulimit -n)**, memory per connection, and kernel buffers.

1.8 Multithreading vs Async systems

Feature	Multi-threading	Async / event loop
Concurrency source	OS threads	Callback / microtask queue
Parallelism	✓ Yes (multi-core)	✗ Not on its own — one thread (unless workers/processes are added)
Scheduling	OS preemptive	Cooperative (tasks yield at <code>await</code>)
Memory per task	~0.5–1 MB stack reserved per thread (varies by OS/runtime)	A few KB – tens of KB per connection (objects/closures)
Switch cost	High (kernel context switch, μ s + cache misses)	Very low (function call, ns)
Shared-memory bugs	Data races, deadlocks, need locks	No data races on JS variables (see correction below)
Best for	CPU-heavy tasks, true parallel compute	I/O-heavy tasks, massive connection counts
Main failure mode	Race conditions, deadlock, thread explosion	Blocking the event loop

🔗 CORRECTION TO YOUR ORIGINAL NOTE — “No race conditions on JS variables”

- True for *data races*: only one JS thread touches memory at a time, so two threads can never corrupt the same variable mid-write.
- But **logical race conditions still exist** across `await`. Example: `if (stock > 0) { await db.save(); stock--; }` — two requests can both pass the check while the first awaits. Fix with atomic DB operations (`UPDATE ... WHERE stock > 0`), transactions, or distributed locks.

◆ SENIOR INSIGHT — Hybrid models used in real systems

- **Go**: goroutines (M:N scheduling — millions of lightweight tasks over a few OS threads). **Java 21**: virtual threads. **Erlang/Elixir**: lightweight processes + message passing.
- **Nginx**: several worker *processes*, each with its own event loop. **Node**: event loop + libuv thread pool + optional worker threads / cluster.
- Pattern to remember: **event loop for waiting, threads/processes for computing**.

1.9 Node.js internals: V8 + libuv

Node.js is not “just JavaScript”. It is a runtime built from cooperating parts:

+-----+ Your JavaScript code +-----+		
Node.js core APIs (fs, http, crypto, timers, streams) +-----+		
V8 engine (Google)	libuv (C library)	
parse > compile >	event loop, thread pool,	
machine code, heap,	async I/O, timers, DNS,	
call stack, GC	child processes	
+-----+		
Operating system (epoll / kqueue / IOCP, threads, files) +-----+		

V8 — the JavaScript execution engine

- Parses → compiles (JIT) → generates machine code; runs JS **synchronously** on one call stack.
- Manages memory: heap + **garbage collection**. Also implements Promises and the microtask queue.
- **Does not provide** timers, file I/O, networking or the event loop — the host (Node/browser) provides those.

libuv — the asynchronous engine

- Cross-platform C library: **event loop, thread pool**, async network I/O, timers, file-system operations, DNS, child processes, signals.
- Hides OS differences (epoll vs kqueue vs IOCP) behind one API.
- Without libuv, Node.js could not do I/O.

🔗 CORRECTION TO YOUR ORIGINAL NOTE — “V8 has no async operations”

- More precise: V8 itself supports Promises/async-await syntax and runs the microtask queue, but it has **no timers, I/O or event loop**. Those come from libuv. Memory aid: **“V8 runs, libuv waits.”**

What happens on `fs.readFile(path, cb)`

1. JS calls `fs.readFile` → V8 hands the request to Node → libuv
2. libuv queues the (blocking) read onto a thread-pool thread
3. JS thread is FREE: keeps running other callbacks / requests
4. Thread pool finishes the read → libuv places 'cb' in the event-loop queue
5. Event loop picks 'cb' → V8 executes it on the single JS thread

★ MUST REMEMBER — Event-driven = “Don’t wait”

- You register a callback (or await a promise), continue executing, and the system notifies you later.
- Benefit: no blocking, memory-cheap, handles very large connection counts — **for I/O-bound work** (qualified: also needs fd limits and kernel tuning).

1.10 The libuv thread pool (a favourite interview topic)

Not everything in Node is done by the event-loop thread alone. Operations that the OS cannot do in a non-blocking way are sent to a pool of worker threads.

Uses the thread pool	Does NOT use the thread pool
fs.* (file system; regular-file I/O has no portable non-blocking API) crypto : pbkdf2, scrypt, randomBytes zlib compression dns.lookup (uses getaddrinfo)	TCP / UDP / HTTP sockets (use epoll/kqueue/IOCP directly) Pipes, TTY dns.resolve* (uses c-ares, network based) Timers (handled by the loop itself) Promises / microtasks (pure JS)

★ MUST REMEMBER

- Default pool size = **4** threads (max 1024). Configure with `UV_THREADPOOL_SIZE`.
- Pool size does not depend on CPU cores. Extra tasks simply **queue** and wait.

Your example, explained

```
const crypto = require('crypto');
const start = Date.now();
// process.env.UV_THREADPOOL_SIZE = 6;    // see warning below

crypto.pbkdf2('password1', 'salt1', 100000, 1024, 'sha512', () => {
  console.log(`password1 took ${Date.now() - start} ms`);
});
crypto.pbkdf2('password2', 'salt2', 100000, 1024, 'sha512', () => {
  console.log(`password2 took ${Date.now() - start} ms`);
});
```

Scenario (each hash ≈ T ms of CPU)	Expected behaviour (illustrative)
Default pool = 4, 2 tasks, ≥2 free cores	Both finish at ≈ T — truly parallel (two threads on two cores)
<code>UV_THREADPOOL_SIZE=1</code>	First ≈ T, second ≈ 2T — tasks queue behind one thread
Default pool = 4, 5 tasks	First four ≈ T together; fifth ≈ 2T (waits for a free thread)
Pool = 8 but machine has 4 cores	Threads compete for cores → context switching, no real speed-up for CPU-bound work

✗ COMMON MISTAKE — Setting the pool size inside the script

- The pool is created lazily at first use; `process.env.UV_THREADPOOL_SIZE = n` set too late (or on some platforms) is ignored. Prefer: `UV_THREADPOOL_SIZE=8 node app.js`.

◆ SENIOR INSIGHT — Thread-pool starvation

- Slow `fs/crypto/dns.lookup` calls can fill all 4 threads, so **unrelated** file reads and DNS lookups queue behind them — latency rises while CPU looks idle.
- Diagnose: event-loop lag normal but `fs/crypto/DNS` latency high. Fix: raise pool size (≈ cores for CPU-bound `crypto`), move heavy work to worker threads, cache DNS.

1.11 Event loop in depth

The **event loop** is a loop that: **(1)** picks a task/callback, **(2)** runs it to completion, **(3)** drains microtasks (promises), **(4)** repeats. It runs on a **single thread** but supports concurrency through **non-blocking I/O**. Because a callback runs to completion, nothing can interrupt it — and nothing else can run until it returns.

The six phases of Node's loop (libuv)

#	Phase	What runs here
1	Timers	Callbacks of <code>setTimeout</code> / <code>setInterval</code> whose time has elapsed (never earlier; may be later)
2	Pending callbacks	Deferred system callbacks (e.g., certain TCP errors)
3	Idle, prepare	Internal use
4	Poll	Retrieves new I/O events and runs I/O callbacks. If nothing else is scheduled, the loop blocks here waiting for events (this is where it sleeps efficiently)
5	Check	<code>setImmediate</code> callbacks — run right after poll
6	Close callbacks	e.g., <code>socket.on('close')</code>

Between **every** callback (and between phases) Node drains **two** queues in this order: **process.nextTick queue** first, then **Promise microtasks**.

```
console.log('A sync');
setTimeout(() => console.log('D timeout 0'), 0);
setImmediate(() => console.log('E immediate'));
Promise.resolve().then(() => console.log('C promise'));
process.nextTick(() => console.log('B nextTick'));

// Output (CommonJS): A sync, B nextTick, C promise, then D / E
// D vs E order from the main module is NOT guaranteed;
// inside an I/O callback, setImmediate (E) always runs before setTimeout (D).
```

★ **MUST REMEMBER** — Priority order to memorise

- **Synchronous code** → **nextTick** → **Promise microtasks** → **timers / I/O / setImmediate (by phase)**
- Mnemonic for phases: **T-P-I-P-C-C** = *Timers, Pending, Idle, Poll, Check, Close*.

✗ **COMMON MISTAKE** — Starving the loop with microtasks

- A recursive `process.nextTick` or endless promise chain never lets the loop reach the poll phase — I/O is starved. Use `setImmediate` to yield to the loop.

Browser vs Node: the browser's event loop is defined by the HTML standard and shares the **main thread with UI rendering** (tasks, microtasks, `requestAnimationFrame`, `paint`). Node's is implemented by libuv with the phases above and has no rendering. Same concept, different implementation. Simulation tool: jsflow.info.

1.12 Blocking the event loop — the #1 Node.js production problem

Because there is **one** JS thread, any long synchronous task delays **every** connected user (all timers, all responses).

Typical causes	Symptoms	Fixes
Big <code>JSON.parse/stringify</code> Heavy loops, sorting huge arrays Sync APIs (<code>readFileSync</code> , <code>pbkdf2Sync</code>) Catastrophic regex (ReDoS) Image/PDF processing in-process	Latency spikes for all requests Health checks time out CPU 100% on one core while others idle Event-loop lag metric rises	Worker threads or child processes for CPU work Offload to a job queue (Ch. 12) Chunk work with <code>setImmediate</code> Use streaming instead of loading whole payloads Never use <code>*Sync</code> in servers

◆ SENIOR INSIGHT — What to monitor

- **Event-loop lag/delay** (e.g., `perf_hooks.monitorEventLoopDelay`), p99 latency, CPU per process, active handles, thread-pool queue length.
- Rule of thumb: keep each callback under a few milliseconds. If a request needs more than ~10-50 ms of pure CPU, it does not belong on the main thread.

1.13 Using all cores: cluster vs worker threads vs child processes

Node runs your JS on one thread, so a 16-core server uses ~1 core unless you scale deliberately.

Tool	What it creates	Memory	Best for
cluster	Multiple Node processes (one per core) all listening on the same port; master distributes connections (round-robin by default on Linux)	Isolated; no shared state — keep sessions in Redis/DB	Scaling an HTTP server across cores
worker_threads	Extra threads in the same process, each with its own V8 isolate + event loop	Message passing (<code>postMessage</code>) or <code>SharedArrayBuffer</code> + <code>Atomics</code>	CPU-heavy tasks (image resize, parsing, crypto)
child_process	Separate OS process, possibly a different program	Isolated; IPC pipes	Running external tools/scripts (ffmpeg, Python)
Process manager / orchestrator	PM2, systemd, Docker/Kubernetes replicas behind a load balancer	Isolated	Production: one container per core or per pod + auto-restart

◆ SENIOR INSIGHT — Modern production practice

- Most teams run **1 Node process per container** and let Kubernetes + a load balancer scale replicas, instead of using `cluster` inside the container.
- Because workers are separate processes, **in-memory state (sessions, caches, rate-limit counters) is not shared** — this is the first step toward stateless services and external caches (Ch. 7 & 11).

1.14 Real-world example: 1000 clients requesting data

	Thread-per-request server	Async server (Node.js / Nginx)
Model	1000 threads, each blocks waiting for the DB	1 thread (per process) handles all requests; DB calls are async
Memory	High ($\approx 1 \text{ MB stack} \times 1000 \approx \sim 1 \text{ GB reserved}$)	Low (small per-connection state)
CPU overhead	High context switching and scheduling	Minimal; loop only runs ready callbacks
Strength	Heavy computation per request	Many waiting connections, extreme scalability
Weakness	Doesn't scale to tens of thousands of connections	One CPU-heavy request stalls everyone

Important: in both models the real bottleneck is often the **database**, not the web server. 1000 concurrent requests still need 1000 DB operations — so connection pools, caching, replicas and queues (later chapters) matter more than the choice of web server.

1.15 Decision guide

Situation	Choose	Why
REST API / chat / gateway with lots of waiting on DB & network	Async event loop (Node, Nginx, asyncio, Netty)	Cheap concurrency for I/O-bound work
Video encoding, ML inference, big data transforms	Threads / processes (one per core), or specialised workers	CPU-bound; needs true parallelism
Mostly I/O but a few heavy tasks	Event loop + worker threads or job queue	Keeps the loop responsive
Need strong isolation / security	Separate processes or containers	Memory isolation limits blast radius
Very high concurrency + simple code	Go goroutines / Java virtual threads	Looks like blocking code, scales like async

1.16 Interview Q&A (answer in 30 seconds)

Q1. Node is single-threaded, so how does it handle 10k connections?

Network I/O is non-blocking via epoll/kqueue. The single JS thread only runs short callbacks; waiting is done by the OS/libuv, so no thread is parked per connection.

Q2. What is the libuv thread pool and what uses it?

A pool of 4 threads (default) for work with no non-blocking OS API: fs, crypto (pbkdf2/scrypt), zlib, dns.lookup. Sockets do not use it.

Q3. Concurrency vs parallelism?

Concurrency = overlapping progress on many tasks (can be 1 core). Parallelism = simultaneous execution (needs multiple cores).

Q4. How do you run a CPU-heavy task in Node?

Move it off the event loop: worker_threads, child process, or a queue + worker service. Never run it in a request handler.

Q5. Process vs thread?

Process = isolated memory, heavy; thread = shares memory inside a process, light but needs synchronization.

Q6. Why is context switching expensive?

Saving/restoring state plus losing warm CPU caches/TLB, and kernel involvement; it does no useful work.

Q7. nextTick vs Promise vs setImmediate vs setTimeout(0)?

Order: sync → nextTick → promise microtasks → timers/immediate by loop phase. setImmediate runs in the check phase after poll.

Q8. When would you prefer multi-threading over an event loop?

CPU-bound work needing true parallelism and shared memory (e.g., in-memory analytics, game servers) where the cost of locks is acceptable.

Q9. How do you use all cores of a Node server?

Run multiple processes (cluster / PM2 / container replicas behind a load balancer); keep services stateless; use worker threads for CPU tasks.

Chapter 1 Cheat Sheet

⚡ QUICK RECALL — Revise this in 2 minutes

- **Core** = 1 thread at a time. **Process** = isolated container. **Thread** = shared heap, own stack.
- **Shared memory** ⇒ **race conditions** ⇒ **locks** ⇒ **deadlocks**. Deadlock needs 4 conditions; fix with lock ordering.
- **Context switch** = save/load state + cache loss. Threads > cores ⇒ slower.
- **Preemptive** (OS decides) vs **Cooperative** (task yields at await).
- **Concurrency** = structure / interleaving. **Parallelism** = simultaneous. Amdahl: serial part limits speed-up.
- **I/O-bound** → **async**. **CPU-bound** → **threads/processes** ≈ **cores**.
- **V8 runs JS, libuv waits**. libuv = event loop + thread pool (4) + async I/O.
- Thread pool: fs, crypto, zlib, dns.lookup. **Not** sockets.
- Order: **sync** → **nextTick** → **promises** → **timers** → **poll(I/O)** → **check(setImmediate)** → **close**.
- **Never block the event loop**. Use workers / queues for heavy CPU.
- Scale Node: cluster / replicas behind LB; keep processes stateless.

★ MUST REMEMBER — Numbers worth memorising

- L1 cache ≈ 0.5 ns • main memory ≈ 100 ns • mutex lock/unlock ≈ 25 ns • context switch ≈ μs range • datacenter round trip ≈ 500 μs • SSD random read ≈ 150 μs • HDD seek ≈ 10 ms • cross-continent round trip ≈ 150 ms.
- (Full latency table and estimation practice are in Chapter 2.)